

Diseño y Análisis de Algoritmos

Ayudantía 4: Técnicas de Diseño de Algoritmos

Ayte. Cristian Nettle

1. Count Zeroes

Dado un string binario S de la forma $\{1^m 0^n\}$, diseña un algoritmo que determine la cantidad de ceros en $O(\log k)$ tiempo, donde $k = n + m$.

Considerar que:

- (i) n y m son mayores que 0.

Ejemplos

E.1 $S = "11000"$ $\Rightarrow 3$

E.2 $S = "11110"$ $\Rightarrow 1$

2. Sorting with Few Inversions

Suponga que se conoce que el número de inversiones en un arreglo de tamaño N es 10, donde $10 \ll N$. Las inversiones se definen como pares (i, j) tales que $i < j$ y $A_i > A_j$.

Determine qué algoritmo de ordenamiento debería usarse para ordenar el arreglo completamente entre: Merge Sort, Selection Sort, o Insertion Sort. Justifique adecuadamente su elección.

Considerar que:

- (i) el número de inversiones es mucho menor que N .
- (ii) se busca aprovechar esta condición para mejorar la eficiencia.

3. Interval Scheduling

Se tienen n intervalos o solicitudes $R = \{1, 2, \dots, n\}$. Cada solicitud i requiere el uso exclusivo de un recurso en el intervalo de tiempo $[s(i), f(i)]$, donde $s(i)$ es el tiempo de inicio y $f(i)$ el tiempo de término, con $s(i) < f(i)$.

Dos solicitudes son compatibles si sus intervalos no se solapan, es decir,

$$f(i) \leq s(j) \quad \text{o} \quad f(j) \leq s(i).$$

Problema: Encontrar el subconjunto más grande posible de solicitudes mutuamente compatibles (es decir, el máximo número de intervalos que no se traslapan).

Considerar que:

- (i) se pueden ordenar los intervalos según sus tiempos de término.
- (ii) se puede utilizar un algoritmo voraz para seleccionar intervalos compatibles.

Ejemplos

E.1 Intervalos = $[(1, 3), (2, 5), (4, 6)]$ $\Rightarrow \{(1, 3), (4, 6)\}$

E.2 Intervalos = $[(1, 2), (3, 4), (0, 6), (5, 7), (8, 9)] \Rightarrow \{(1, 2), (3, 4), (5, 7), (8, 9)\}$